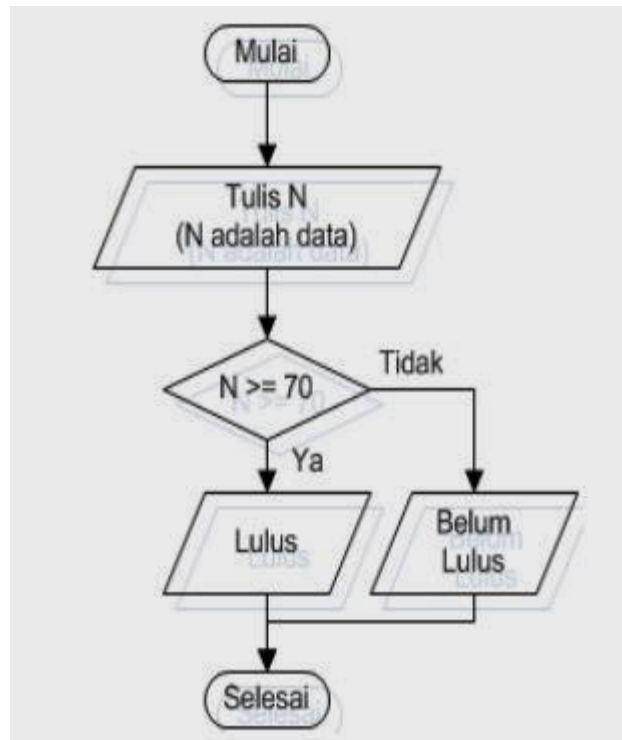


Pertemuan 4

STRUKTUR KONTROL

4.1 Percabangan

Percabangan merupakan istilah yang digunakan untuk menyebut alur program yang bercabang. Percabangan juga dikenal sebagai “Control Flow”, “Struktur Kondisi”, “Decision”, dsb. Semuanya itu sama. Contoh alur program sederhana dengan percabangan:

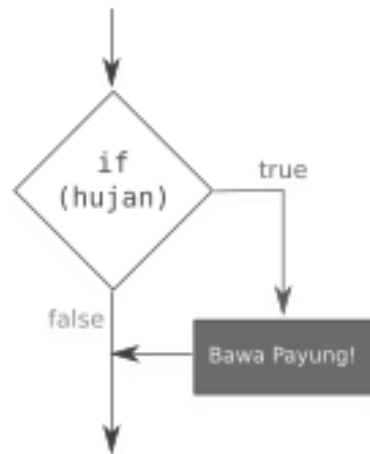


Tiga bentuk percabangan pada Java:

1. Percabangan IF
2. Percabangan IF/ELSE
3. Percabangan IF/ELSE/IF dan SWITCH/CASE

4.1.1 Percabangan IF

Percabangan ini hanya memiliki satu pilihan, yaitu pilihan didalam IF hanya akan dikerjakan jika kondisinya benar, jika salah tidak akan melakukan apa-apa.



```

if (hujan) {
    System.out.println("Bawa Payung!");
}

```

Studi Kasus IF :

Ada sebuah toko pakaian, mereka memberikan souvenir berupa paperbag kepada pembeli yang belanja di atas Rp 300.000.

souvenir.java

```
import java.util.Scanner;
```

```
public class souvenir {
```

```
    public static void main(String[] args) {
```

```
        // membuat variabel belanja dan scanner
```

```
        int belanja = 0;
```

```
        Scanner scan = new Scanner(System.in);
```

```
        // mengambil input
```

```
        System.out.print("Total Belanjaan: Rp ");
```

```
        belanja = scan.nextInt();
```

```
        // cek apakah dia belanja di atas 300000
```

```
        if ( belanja > 300000 ) {
```

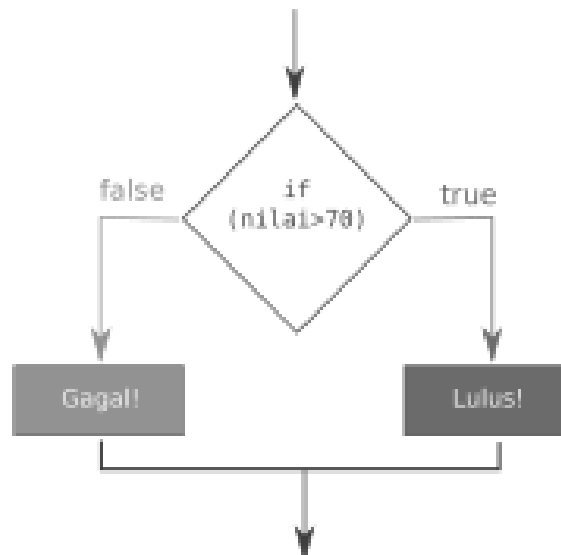
```
            System.out.println("Selamat, anda mendapatkan souvenir paperbag!");
```

```
        }
```

```
    }
```

4.1.2 Percabangan IF/ELSE

Percabangan IF/ELSE memiliki pilihan alternatif jika kondisi benar akan mendapatkan hasil dan kondisi salah akan juga mendapatkan hasil.



```
if (nilai > 70) {  
    System.out.println("Lulus!");  
} else {  
    System.out.println("Gagal!");  
}
```

Studi Kasus IF/ELSE :

Jika nilai mahasiswa lebih besar dari atau sama dengan 70, maka dia dinyatakan lulus. Jika tidak, maka dia tidak lulus.

ceknilai.java

```
import java.util.Scanner;
```

```
public class ceknilai {
```

```
    public static void main(String[] args) {
```

```
        // membuat variabel dan Scanner
```

```
        int nilai;
```

```
        String nama;
```

```
        Scanner scan = new Scanner(System.in);
```

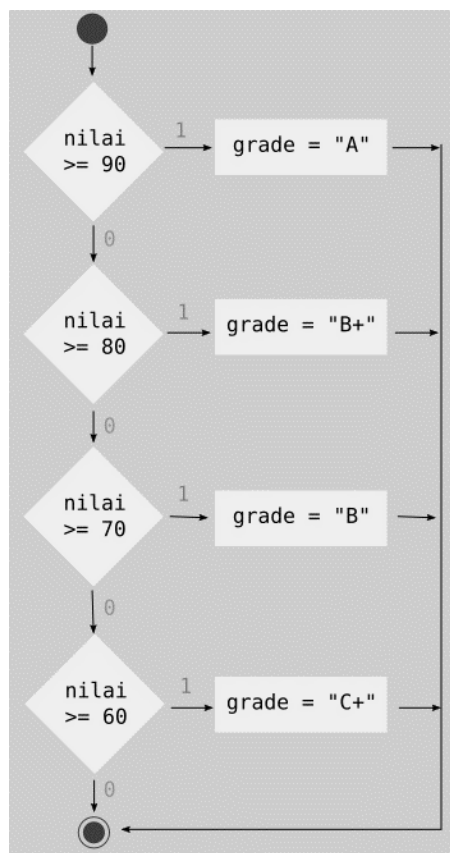
```

// mengambil input
System.out.print("Nama: ");
nama = scan.nextLine();
System.out.print("Nilai: ");
nilai = scan.nextInt();
// cek nilai mahasiswa lulus atau tidak
if( nilai >= 70 ) {
    System.out.println("Selamat " + nama + ", anda lulus!");
} else {
    System.out.println("Maaf " + nama + ", anda tidak lulus!");
}
}
}
}

```

4.1.3 Percabangan IF/ELSE/IF dan SWITCH/CASE

Jika percabangan IF/ELSE hanya memiliki dua pilihan sedangkan percabangan IF/ELSE/IF memiliki lebih dari dua pilihan.



Studi Kasus IF/ELSE/IF :

Jika nilai lebih besar atau sama dengan 90 maka grade "A", jika lebih besar atau sama dengan 80 maka "B+", dan seterusnya.

nilaigrade.java

```
import java.util.Scanner;
public class nilaigrade {
    public static void main(String[] args) {
        // buat variabel dan scanner
        int nilai;
        String grade;

        Scanner scan = new Scanner(System.in);
        // input
        System.out.print("Inputkan nilai: ");
        nilai = scan.nextInt();
        // grade
        if ( nilai >= 90 ) {
            grade = "A";
        } else if ( nilai >= 80 ){
            grade = "B+";
        } else if ( nilai >= 70 ){
            grade = "B";
        } else if ( nilai >= 60 ){
            grade = "C";
        } else if ( nilai >= 40 ){
            grade = "D";
        } else {
            grade = "E";
        }
        // cetak hasil
        System.out.println("Grade: " + grade);
    } }
```

Percabangan SWITCH/CASE mirip seperti percabangan IF/ELSE/IF. Meskipun formatnya berbeda, tetapi cara kerjanya sama. Contoh dalam percabangan switch case:

```
switch (variabel) {  
    case 'A':  
        // pilihan A  
        break;  
    case 'B':  
        // pilihan B  
        break;  
    default:  
        // pilihan C  
}
```

Case A berarti nilai variabel yang akan dibandingkan, apakah nilainya sama dengan A atau tidak. Jika sama, maka kerjakan kode yang ada di dalam case A, begitupun dengan case B dan seterusnya. Break berarti berhenti, ini untuk memerintahkan komputer untuk berhenti mengecek case yang lain. Default berarti jika nilai variabel tidak ada yang sama dengan pilihan case di atas, maka kerjakan kode yang ada di dalam default. Pilihan default bisa juga tidak memiliki break, karena default adalah pilihan terakhir yang berarti pengecekan akan berakhir disitu.

contohswitchcase.java

```
import java.util.Scanner;  
  
public class contohswitchcase {  
    public static void main(String[] args) {  
        // membuat variabel dan Scanner  
        String pelangi;  
        Scanner scan = new Scanner(System.in);  
        // mengambil input  
        System.out.print("Inputkan 1 warna pelangi: ");  
        pelangi = scan.nextLine();
```

```

switch(pelangi){
    case "merah":
        System.out.println("merah (mejikuhibiniu)");
        break;
    case "jingga":
        System.out.println("jingga (mejikuhibiniu)");
        break;
    case "kuning":
        System.out.println("kuning (mejikuhibiniu)");
        break;
    case "hijau":
        System.out.println("hijau (mejikuhibiniu)");
        break;
    case "biru":
        System.out.println("biru (mejikuhibiniu)");
        break;
    case "nila":
        System.out.println("nila (mejikuhibiniu)");
        break;
    case "ungu":
        System.out.println("ungu (mejikuhibiniu)");
        break;
    default:
        System.out.println("bukan warna pelangi!");
}
}
}

```

4.1.4 Operator Logika dalam Percabangan

Operator logika pada percabangan dapat membuat percabangan menjadi lebih singkat.

contohlogikapercabangan.java

```
public class contohlogikapercabangan {  
    public static void main(String[] args) {  
        boolean SIM = false;  
        boolean STNK = true;  
  
        // cek apakah dia akan ditilang atau tidak  
        if(SIM == true && STNK == true){  
            System.out.println("Anda tidak ditilang!");  
        } else {  
            System.out.println("Anda ditilang!");  
        }  
    }  
}
```

Coba modifikasi boolean false dan true nya, lalu dijalankan kembali programnya.

4.2 Perulangan

Perulangan dalam pemrograman terbagi menjadi dua, yaitu:

1. Counted loop: Perulangan yang jumlah pengulangannya terhitung atau tentu, terdiri dari For dan For each
2. Uncounted loop: Perulangan yang jumlah pengulangannya tidak terhitung atau tidak tentu, terdiri dari While dan Do/While

4.2.1 Counted Loop

1. Perulangan For

Format penulisan perulangan For di java:

```
for( int nilai = 0; nilai <= 5; nilai++ ){  
    // blok kode yang akan diulang  
}
```

- variabel hitungan tugasnya untuk menyimpan hitungan pengulangan.

- nilai ≤ 5 artinya selama nilai hitungannya lebih kecil atau sama dengan 5, maka pengulangan akan terus dilakukan sebanyak 10 kali.
- `hitungannya++` fungsinya untuk menambah satu (+1) nilai hitungan pada setiap pengulangan.
- blok kode *For* dimulai dengan tanda '{' dan diakhiri dengan '}'.

contohfor.java

```
class contohfor{
    public static void main(String[] args){
        for(int i=1; i < 5; i++){
            System.out.println("Nama Anda");
        }
    }
}
```

4.2.2 Perulangan For Each

Perulangan ini biasanya digunakan untuk menampilkan isi dari *array* (variabel yang menyimpan lebih dari satu nilai dan memiliki indeks). Perulangan *For Each* dilakukan dengan kata kunci *For*.

contohforeach.java

```
public class contohforeach {
    public static void main(String[] args) {

        // membuat array
        int kelipatanlima[] = {5,10,15,20,25};

        // menggunakan perulangan For each untuk menampilkan angka
        for( int a : kelipatanlima ){
            System.out.print(" "+a);
        }
    }
}
```

4.2.3 Uncounted Loop

1. Perulangan While

Perulangan *While* dapat diartikan *selama*. Cara kerja perulangan ini seperti percabangan, ia akan melakukan perulangan selama kondisi bernilai true. Format penulisan perulangan while:

```
while ( kondisi ) {  
    // blok kode yang akan diulang  
}
```

- *kondisi* bisa diisi dengan perbandingan maupun variabel boolean. *Kondisi* ini hanya memiliki nilai true dan false.
- Perulangan while akan berhenti sampai *kondisi* bernilai false.

contohwhile.java

```
public class contohwhile {  
    public static void main(String[] args) {  
  
        // membuat variabel dan scanner  
        boolean running = true;  
        int ulang = 0;  
        String jawaban;  
        Scanner scan = new Scanner(System.in);  
        while( running ) {  
            System.out.println("Apa anda ingin keluar?");  
            System.out.print("Yes/No : ");  
            jawaban = scan.nextLine();  
        }  
    }  
}
```

```

        // cek jawabannya, kalau ya maka berhenti mengulang
        if( jawaban.equalsIgnoreCase("Yes") ){
            running = false;
        }
        ulang++;
    }
    System.out.println("Anda sudah melakukan perulangan sebanyak " + ulang + " kali");
}
}

```

Jika nilai variabel *running* bernilai *false*, maka perulangan berhenti. Contoh kode *while* di atas dapat dibaca “Lakukan perulangan sebanyak nilai *running* bernilai *true*.” Tetapi bisa juga perulangan ini dapat melakukan *counted loop* seperti *contohwhile2.java*.

contohwhile2.java

```

public class contohwhile2 {
    public static void main(String[] args) {
        int ulang = 1;
        while ( ulang <= 10 ){
            // blok kode yang akan diulang
            System.out.println("Angka ke-" + ulang);

            // increment nilai i
            ulang++;
        }
    }
}

```

2. Perulangan Do/While

Cara kerja perulangan *Do/While* mirip seperti perulangan *While*. Tetapi bedanya *Do/While* melakukan satu kali perulangan, lalu mengecek kondisinya. Format penulisan *Do/While* :

```

do {
    // blok kode yang akan diulang

```

```
} while (kondisi);
```

Kerjakan Do lalu cek kondisi While, jika kondisi bernilai true maka lanjutkan perulangan, jika salah maka berhenti atau keluar.

contohdowhile.java

```
public class contohdowhile {  
    public static void main(String[] args) {  
  
        // membuat variabel  
        int x = 0;  
        do {  
            System.out.println("Mengulang angka dari" + x);  
            x++;  
        } while ( x <= 5);  
    }  
}
```